



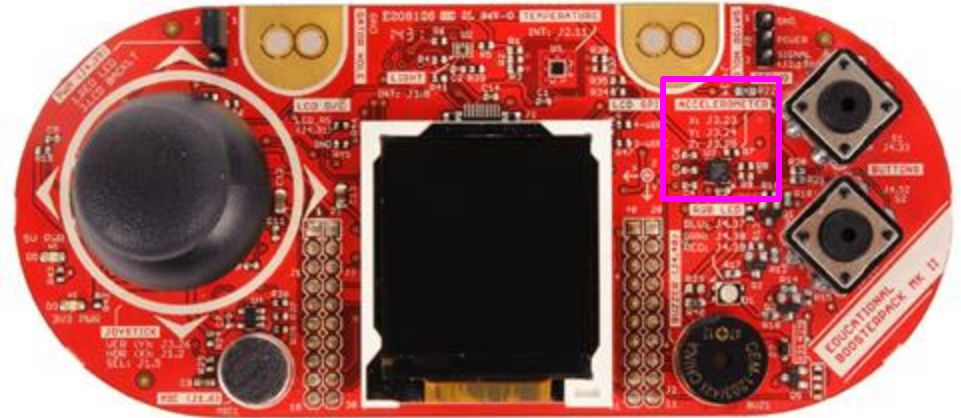
Embedded Software for the Internet Of Things

Accelerometer

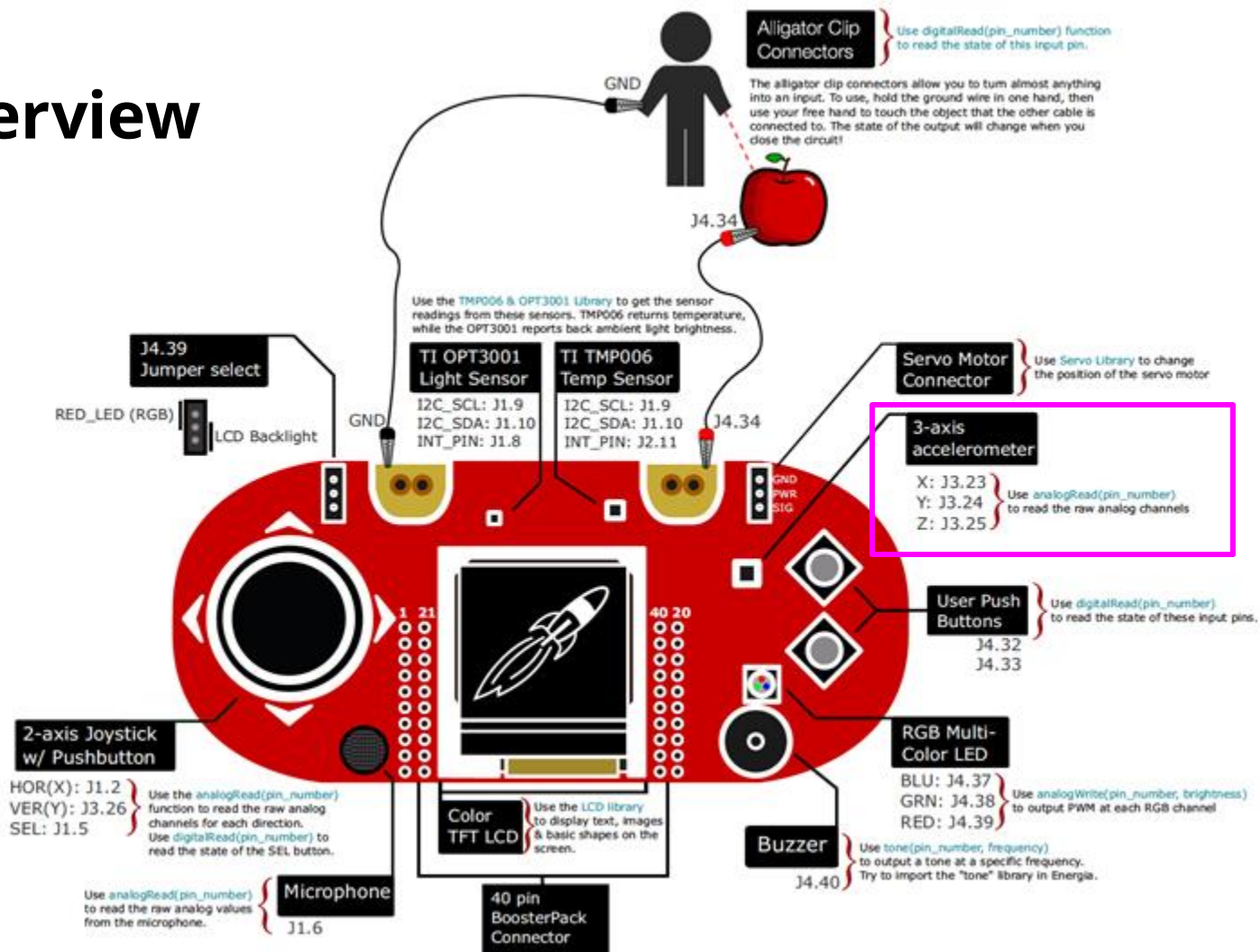
Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

BOOSTXL-EDUMKII EDUCATIONAL BOOSTERPACK

- Various analog and digital inputs and outputs
- TI OPT3001 Light Sensor
- TI TMP006 Temperature Sensor
- Servo Motor Connector
- **3-Axis Accelerometer**
- User Push Buttons
- RGB Multi-color LED
- Buzzer
- Color TFT LCD Display
- Microphone
- 2-Axis Joystick with Pushbutton

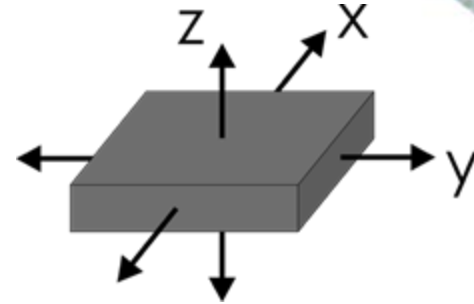


Sensor Overview



Accelerometer

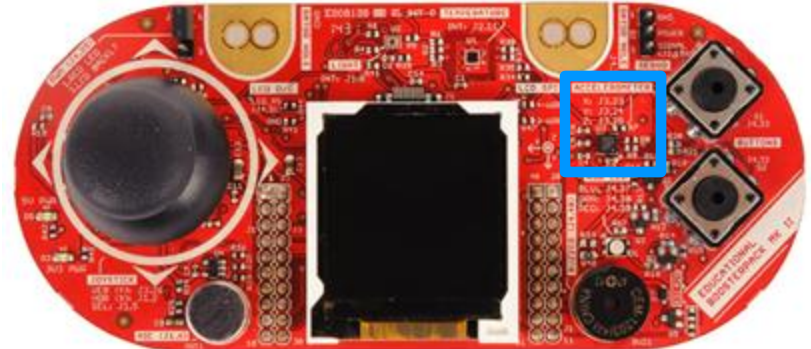
- Accelerometers are **electromechanical** devices that measure acceleration:
 - The rate of change in velocity of an object.
 - How fast something is speeding up or slowing down.



3-AXIS Analog Accelerometer

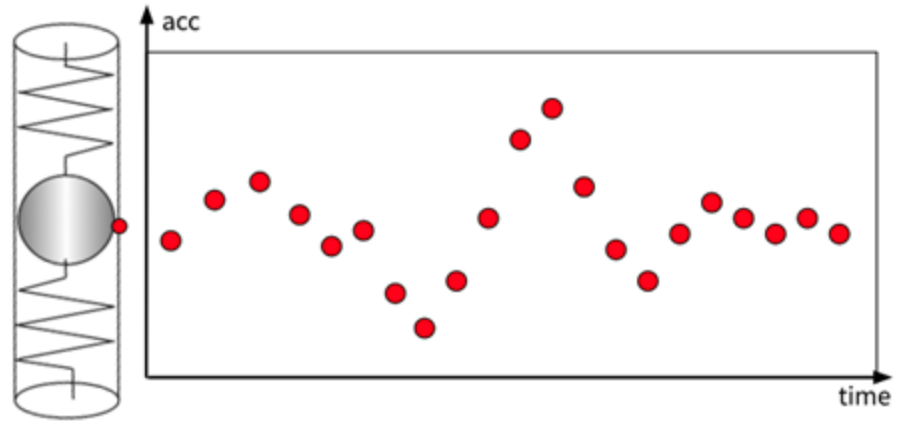
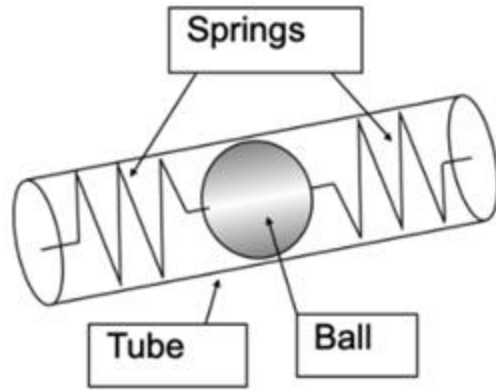
- The **Kionix KXTC9-2050** is a **3-axis** analog accelerometer that measures *g-forces*.
 - Moving the board along the axes will change the analog signal generated by the accelerometer.

BoosterPack Plug-in Module Header Connection	Pin Function
J3.23	Accelerometer X-axis
J3.24	Accelerometer Y-axis
J3.25	Accelerometer Z-axis

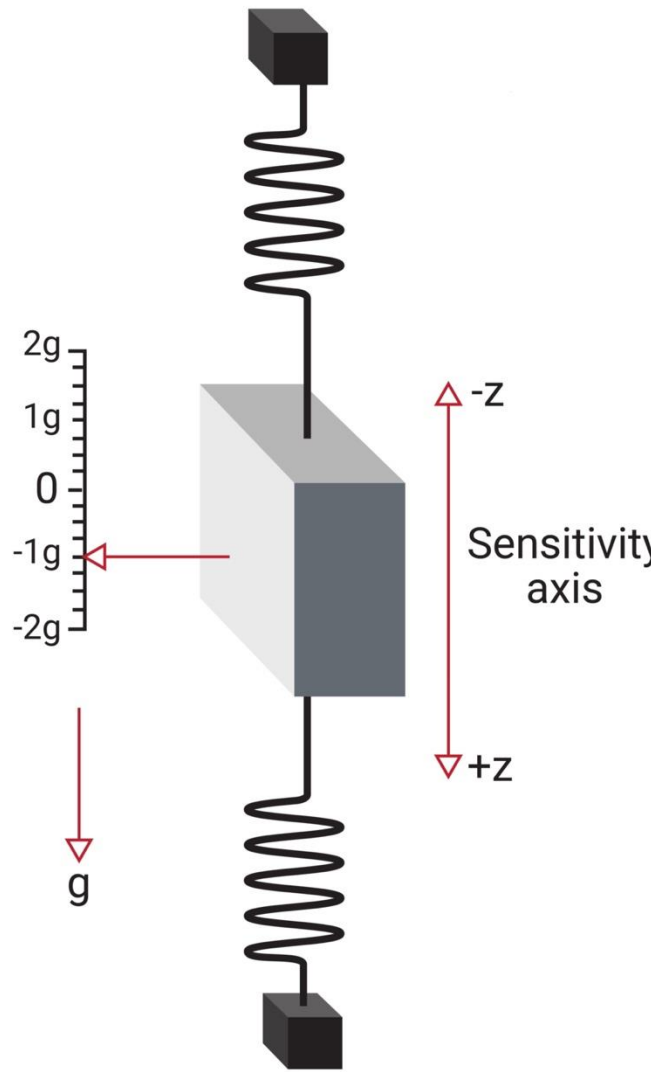
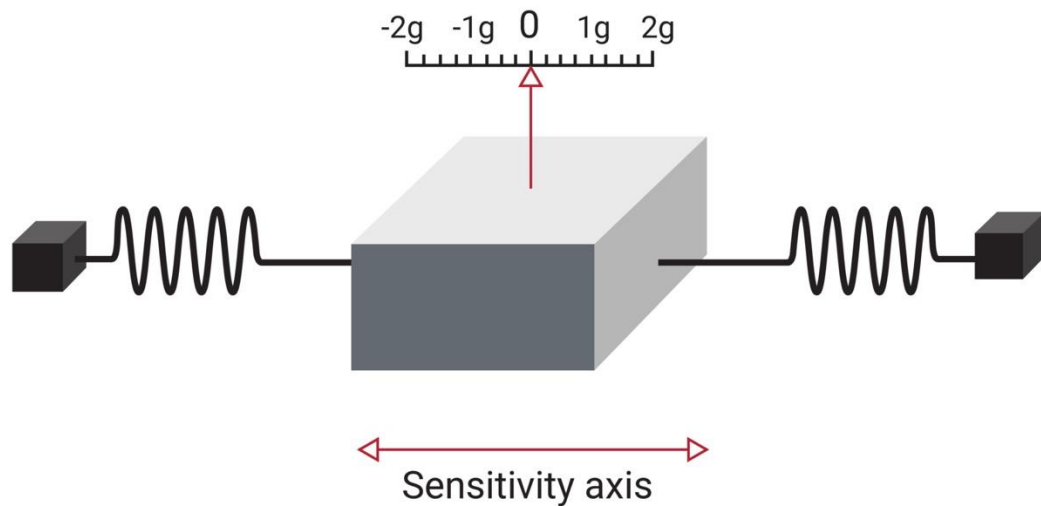


A Simple Accelerometer Model

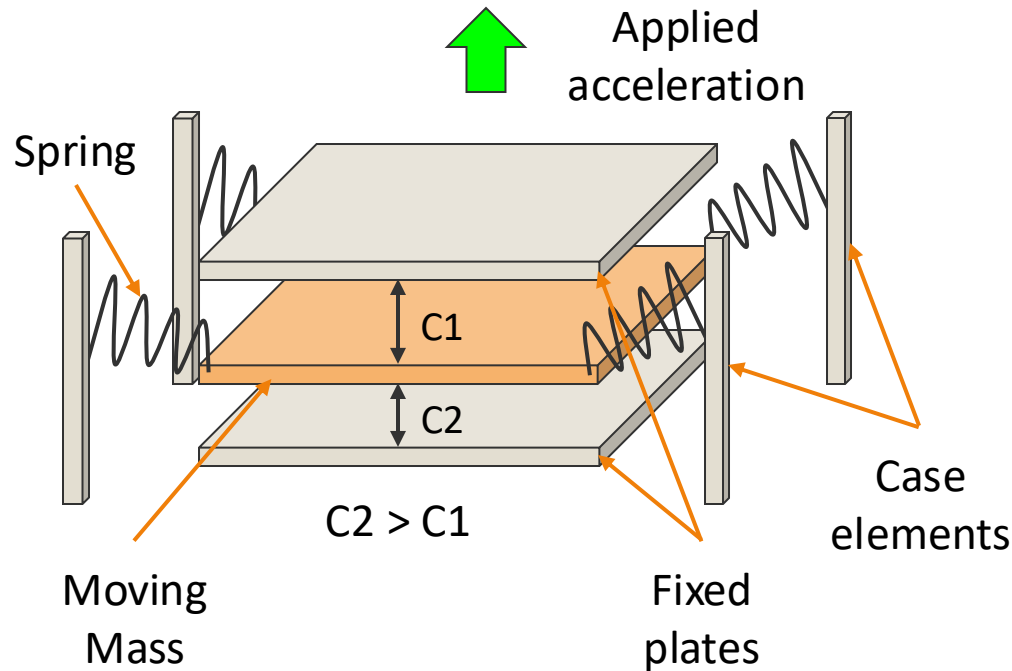
- Acceleration = position of this ball in this tube.



Simple Accelerometer Model

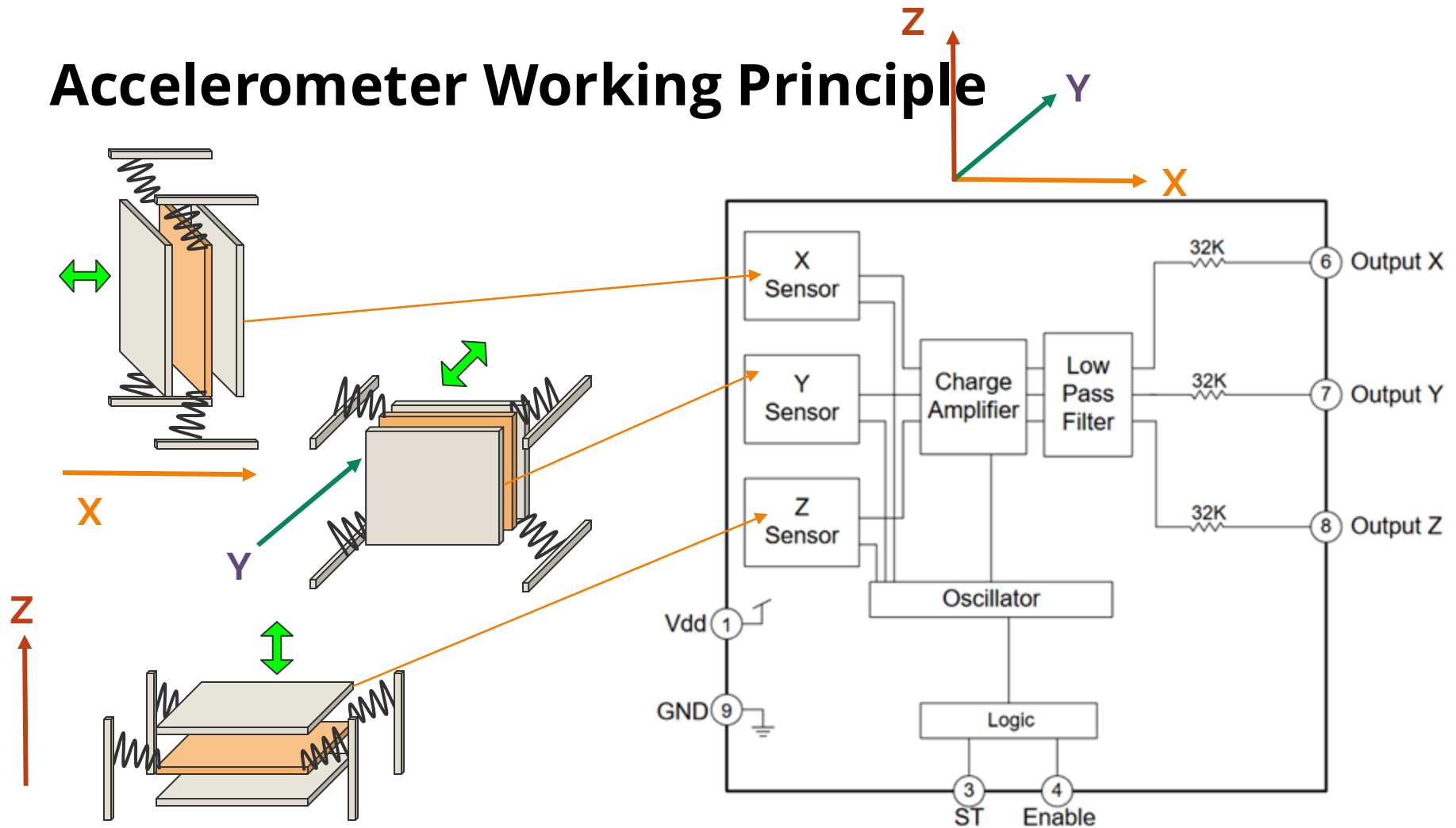


Accelerometer Working Principle



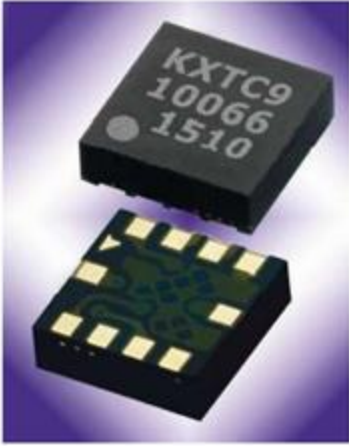
$$V_{out} = \frac{V_{in}}{C_{int}} * (C1 - C2)$$

Accelerometer Working Principle



Accelerometer Output

Maximum **2g** can be measured -> sensors specification.



$$V_{out} = \frac{V_{in}}{C_{int}} * (C1-C2)$$

Parameter	Specification
Zero-g Offset @ RT	1.65 +/- 0.0825 V
Sensitivity @ RT	660 +/- 19.8 mV/g

g: gravitational constant

$$V_{out} = 1.65 \text{ V} \quad \longrightarrow \quad g = 0 = 0 \text{ m/s}^2$$

$$V_{out} = 1.65 + 0.66 \text{ V} \quad \longrightarrow \quad g = 1 = 9.8 \text{ m/s}^2$$

Maximum $\longrightarrow V_{out} = 1.65 + 1.32 \text{ V} \quad \longrightarrow \quad g = 2 = 19.6 \text{ m/s}^2$

$$V_{out} = 1.65 - 0.66 \text{ V} \quad \longrightarrow \quad g = -1 = -9.8 \text{ m/s}^2$$

Minimum $\longrightarrow V_{out} = 1.65 - 1.32 \text{ V} \quad \longrightarrow \quad g = -2 = -19.6 \text{ m/s}^2$

Accelerometer Output

Static X/Y/Z Output Response versus Orientation to Earth's surface (1-g):



Position	1	2	3	4	5	6
Diagram					 Top Bottom	 Bottom Top
X	1.65 V	2.31 V	1.65 V	0.99 V	1.65 V	1.65 V
Y	2.31 V	1.65 V	0.99 V	1.65 V	1.65 V	1.65 V
Z	1.65 V	1.65 V	1.65 V	1.65 V	2.31 V	0.99 V
X-Polarity	0	+	0	-	0	0
Y-Polarity	+	0	-	0	0	0
Z-Polarity	0	0	0	0	+	-

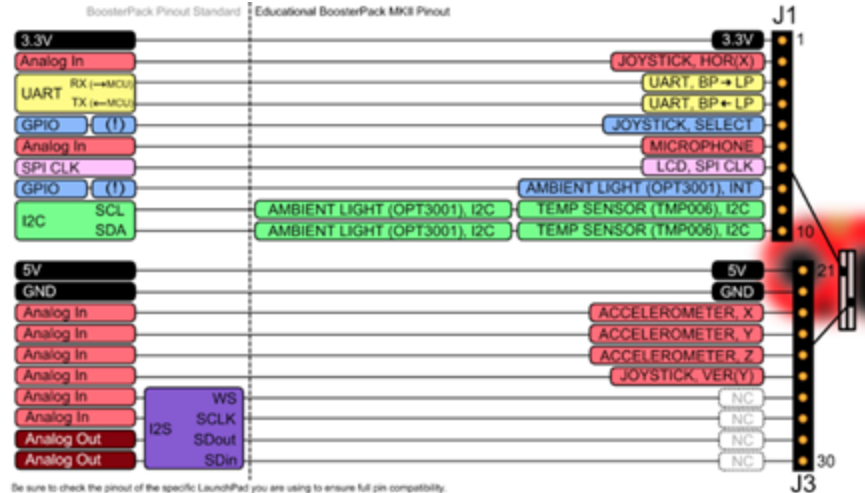
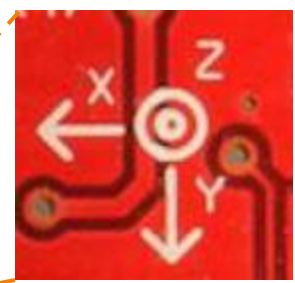
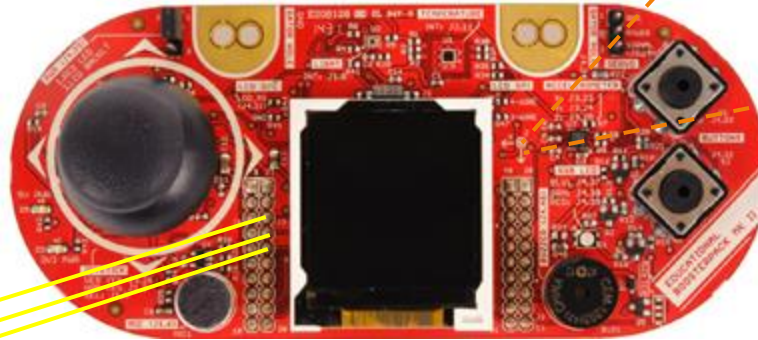
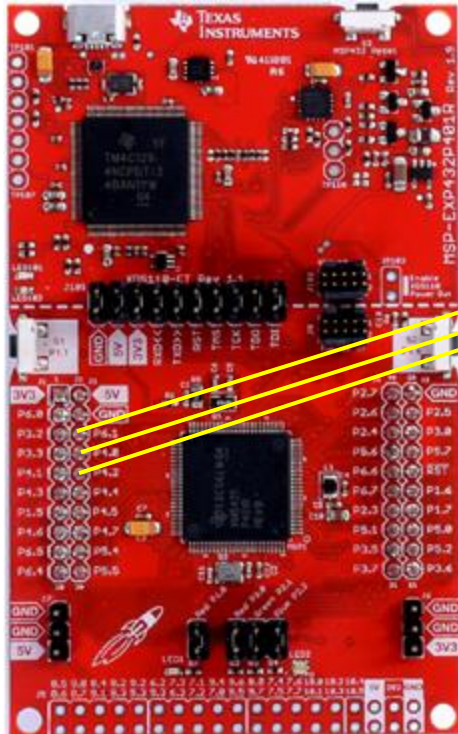


Earth's Surface

Coding Time

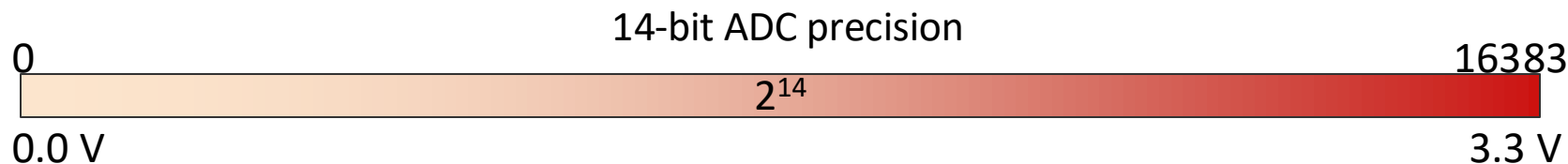
(accelerometer_test.c)

ADC Pins



ADC Translation

ADC_VREFPOS_AVCC_VREFNEG_VSS → Vrefmax = 3.3 V, Vrefmin = 0.0 V



ADC output $\Rightarrow$ 8200 $\longrightarrow$ $8200 / 16383 \approx 0.5$ $\longrightarrow$ $0.5 * 3.3 \text{ V} = \mathbf{1.65 \text{ V}}$

o

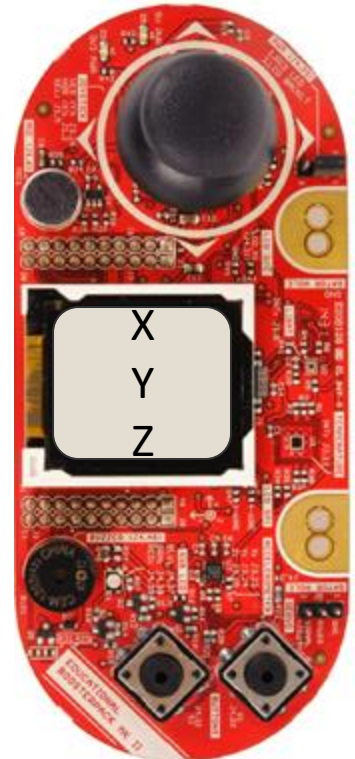
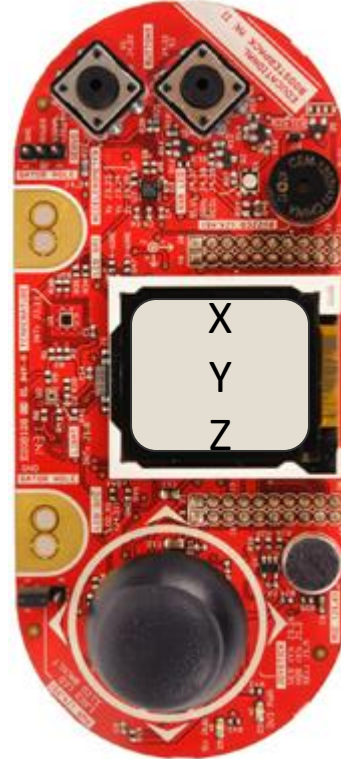
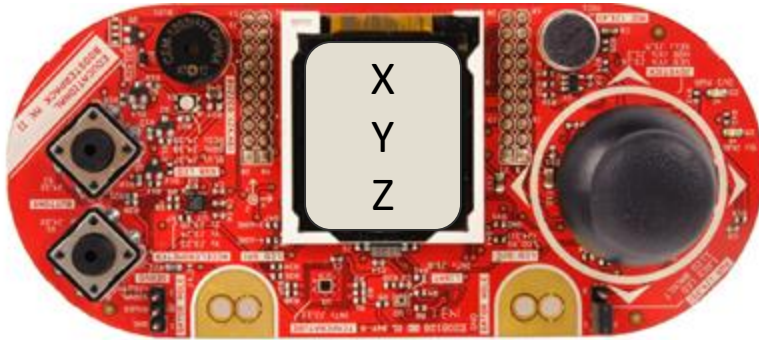
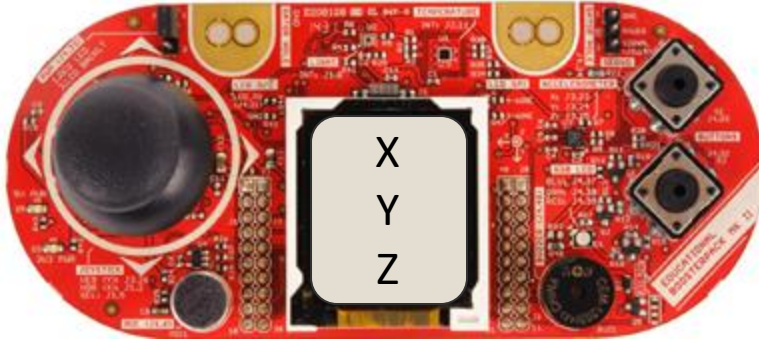
ADC output $\Rightarrow$ 11500 $\longrightarrow$ $11500 / 16383 \approx 0.7$ $\longrightarrow$ $0.7 * 3.3 \text{ V} = \mathbf{2.31 \text{ V}}$

+g

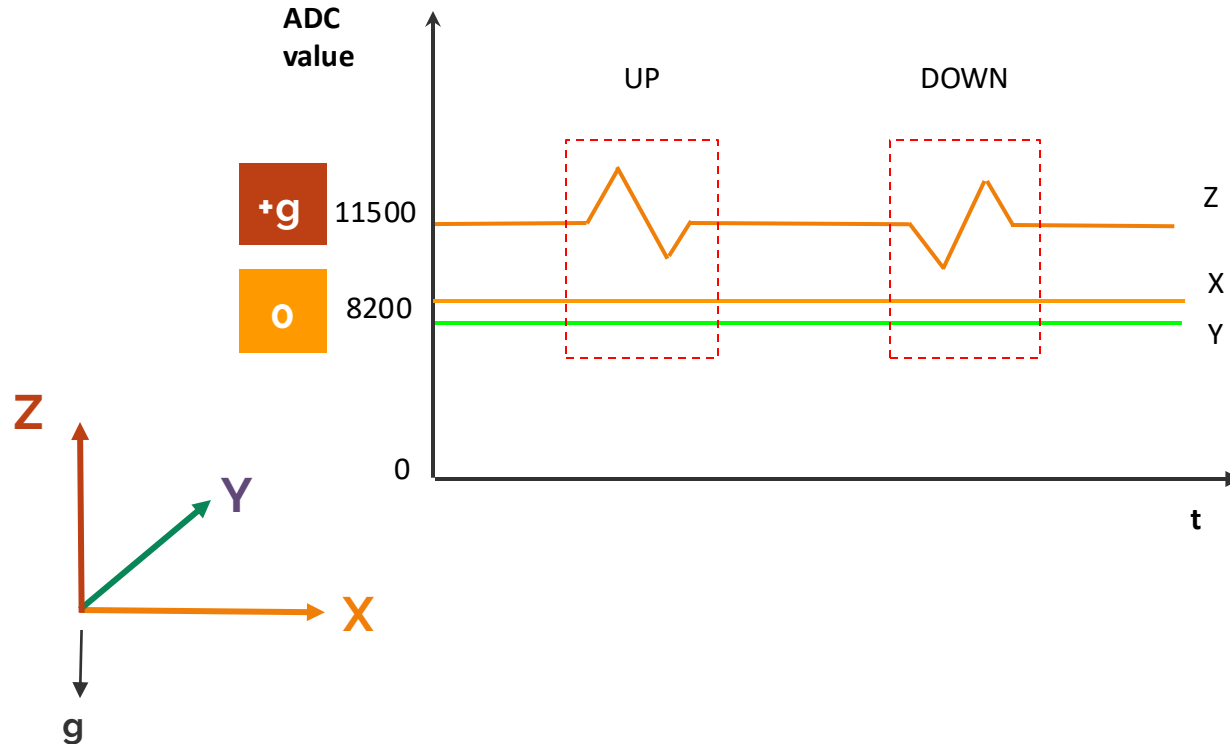
ADC output $\Rightarrow$ 4900 $\longrightarrow$ $4900 / 16383 \approx 0.3$ $\longrightarrow$ $0.3 * 3.3 \text{ V} = \mathbf{0.99 \text{ V}}$

-g

More Coding (accelerometer_lcd.c)



Movement Recognition UP/DOWN (accelerometer_app.c)



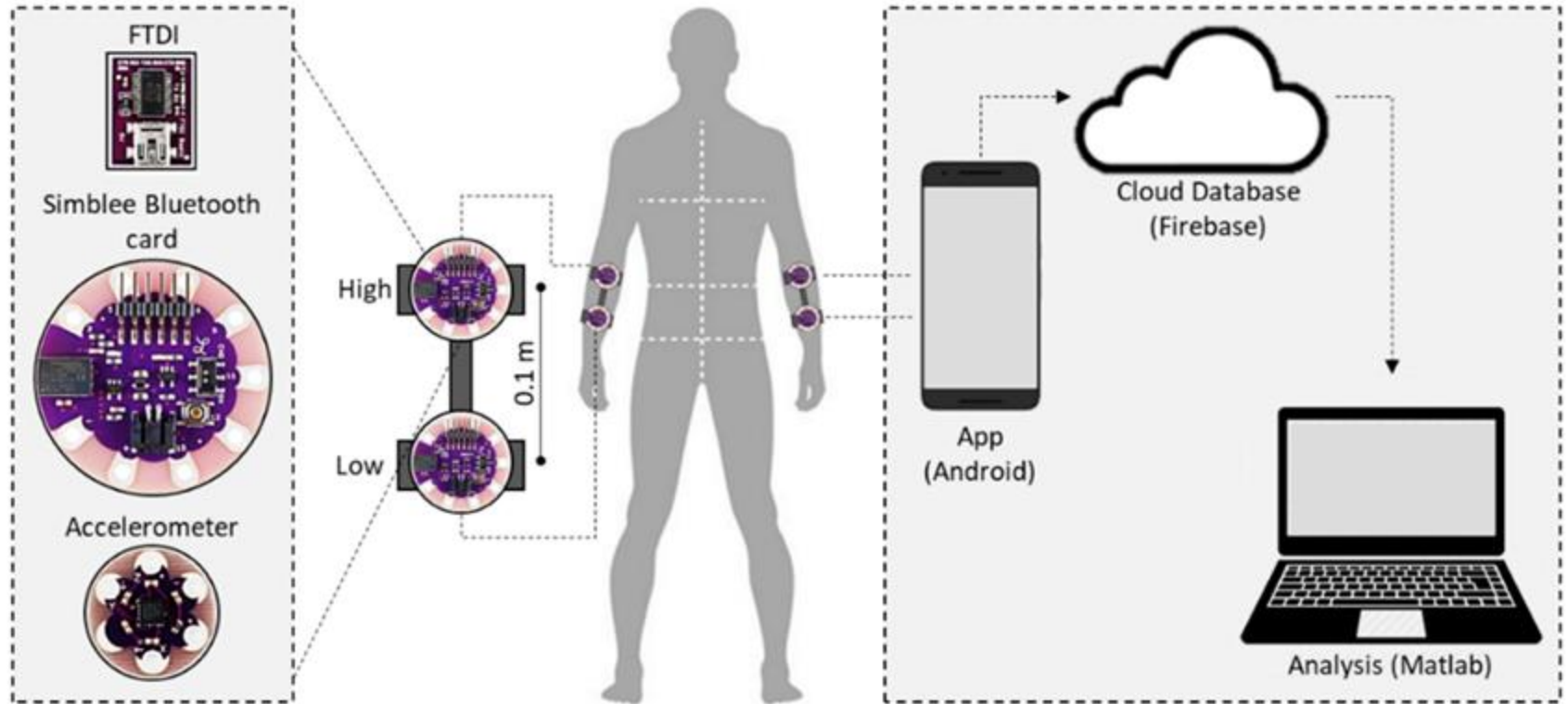
5
Top
Bottom
1.65 V
1.65 V
2.31 V
0
0
+

Your turn!

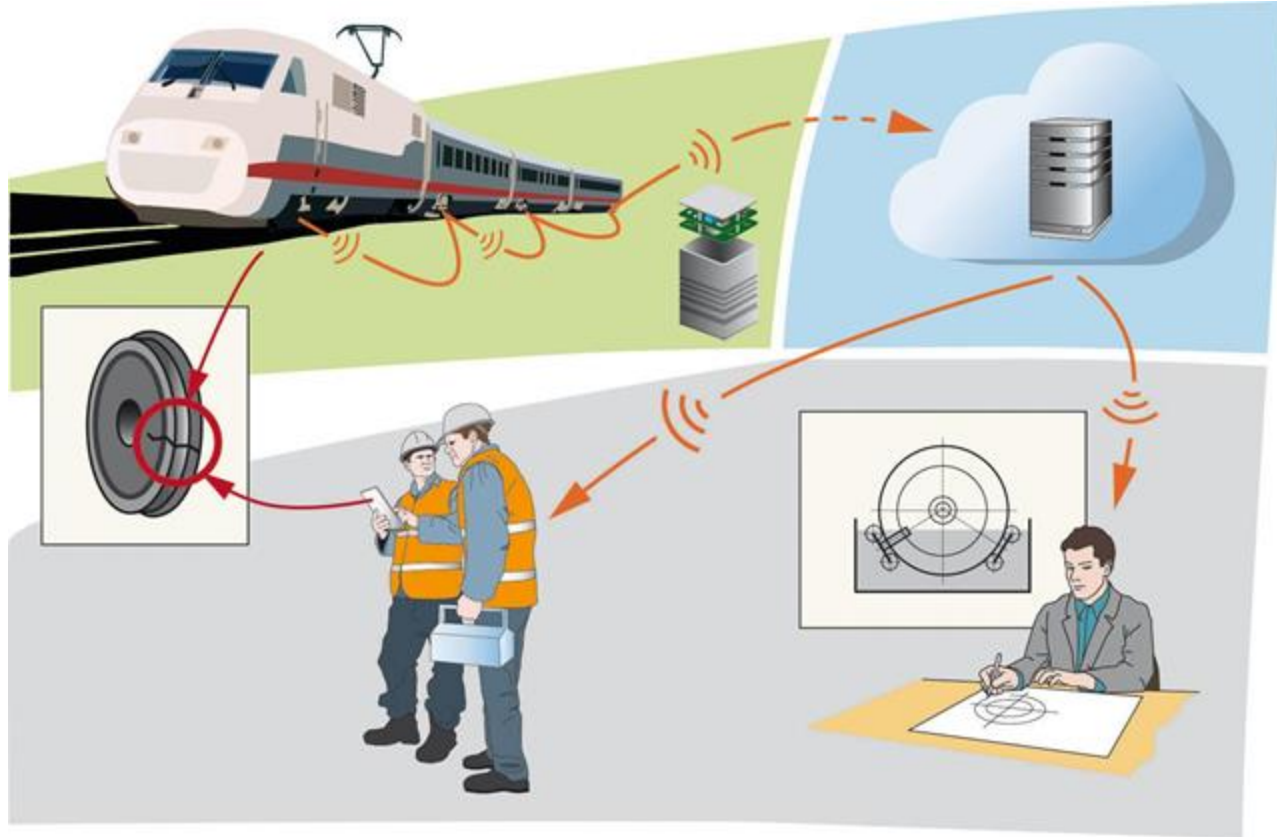
LEFT/RIGHT & FORWARD/BACKWARD

Accelerometers in Cutting-Edge Technologies

Healthcare Innovations



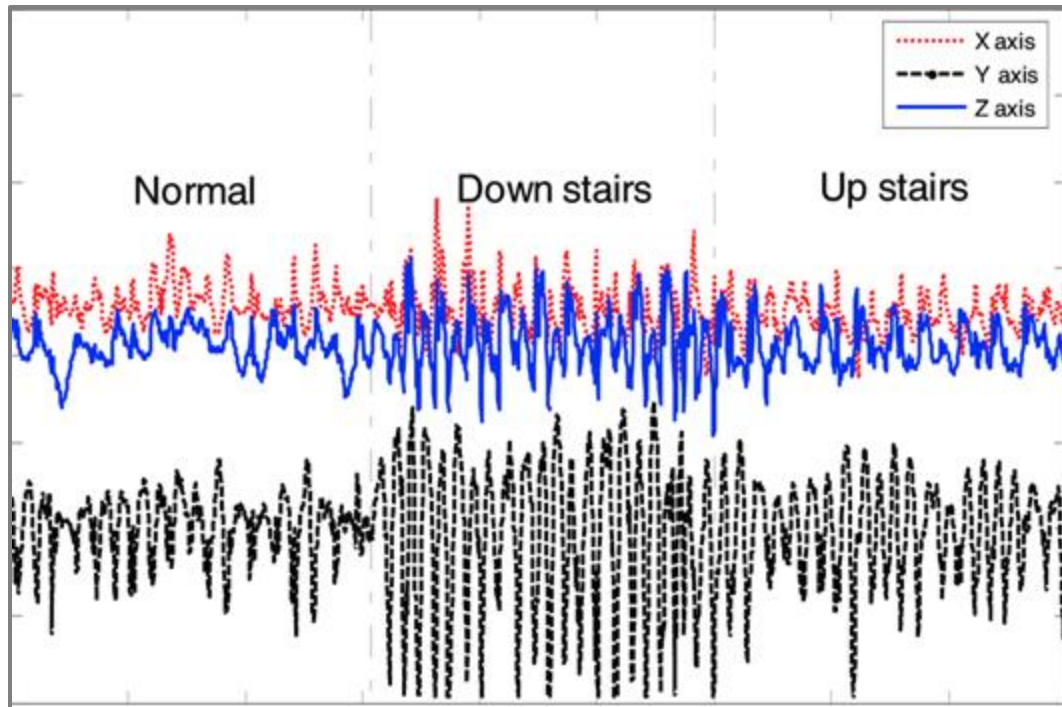
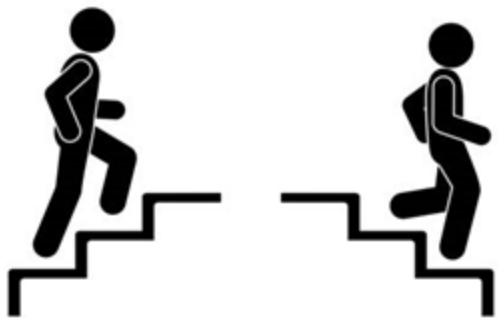
High-Speed Train Monitoring



Part-II

SIMPLE ACTIVITY RECOGNITION

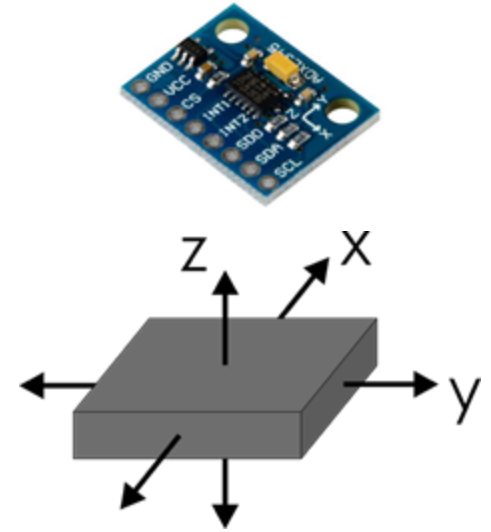
Simple Activity Recognition



Data Structures

- Structure to hold accelerometer sample.

```
typedef struct {  
    uint16_t x;  
    uint16_t y;  
    uint16_t z;  
} threeAxis_t;
```



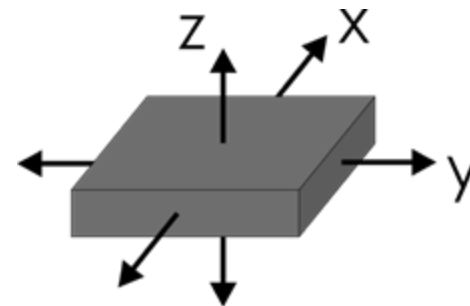
Data Structures

- Structure to hold accelerometer sample.

```
typedef struct {  
    uint16_t x;  
    uint16_t y;  
    uint16_t z;  
} threeAxis_t;
```

- A buffer to hold the samples.

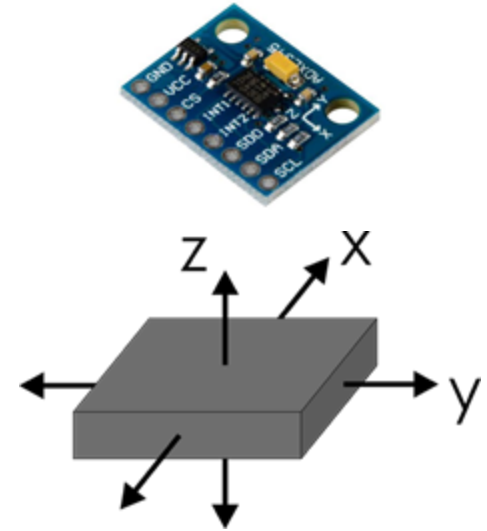
```
#define ACCEL_WINDOW_SIZE 4  
  
typedef threeAxis_t accelReading;  
typedef accelReading accelWindow[ACCEL_WINDOW_SIZE];
```



Reading X, Y and Z Acceleration

- Read sample from accelerometer.
 - The only function that **touches** the hardware!

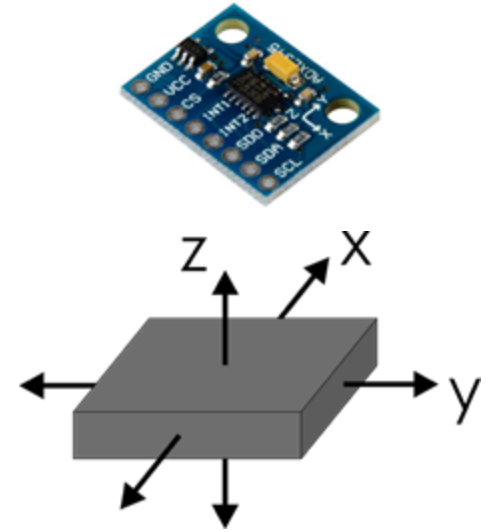
```
void accel_sample(accelReading* result) {  
    result->x = readAccelX();  
    result->y = readAccelY();  
    result->z = readAccelZ();  
}
```



Reading X, Y and Z Acceleration

- Fill the buffer with X, Y and Z samples.

```
void acquire_window(accelWindow window){  
    accelReading sample;  
    unsigned samplesInWindow = 0;  
  
    while (samplesInWindow < ACCEL_WINDOW_SIZE) {  
        accel_sample(&sample);  
        window[samplesInWindow++] = sample;  
    }  
}
```



Classification Constructs

- We are going to represent the collected values using a **mean** and **standard deviation** value.

```
typedef struct {  
    unsigned meanmag;  
    unsigned stddevmag;  
} features_t;
```

Mean of the
magnitude values.

Standard deviation of
the magnitude value.

Classification Constructs

- We are going to represent the collected values using a **mean** and **standard deviation** value.

```
typedef struct {  
    unsigned meanmag;  
    unsigned stddevmag;  
} features_t;
```

Mean of the
magnitude values.

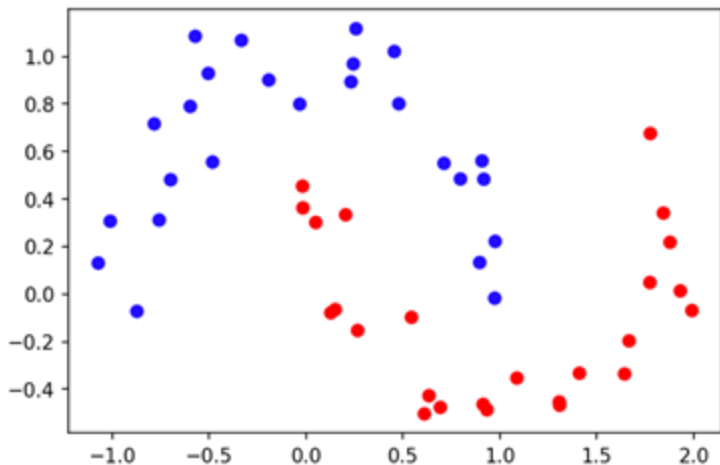
Standard deviation of
the magnitude value.

- We will classify two classes.

```
typedef enum {  
    CLASS_STATIONARY,  
    CLASS_MOVING,  
} class_t;
```

Classification Constructs

- For each class, create a model that represents that class.
 - We will have **16** mean and stddev values to represent each class.



```
#define MODEL_SIZE 16

typedef struct {
    features_t stationary[MODEL_SIZE];
    features_t moving[MODEL_SIZE];
} model_t;
```

Main Program

```
int activity_main()
{
    ...
    model_t model;

    while (1)
    {
        switch (mode) {
            case MODE_TRAIN_STATIONARY:
                train(model.stationary);
                break;
            case MODE_TRAIN_MOVING:
                train(model.moving);
                break;
            case MODE_RECOGNIZE:
                recognize(&model);
                break;
            default:
                break;
        }
    }

    return 0;
}
```

Main Program

- Training!

```
int activity_main()
{
    ...
    model_t model;

    while (1)
    {
        switch (mode) {
            case MODE_TRAIN_STATIONARY:
                train(model.stationary);
                break;
            case MODE_TRAIN_MOVING:
                train(model.moving);
                break;
            case MODE_RECOGNIZE:
                recognize(&model);
                break;
            default:
                break;
        }
    }

    return 0;
}
```

Training

```
void train(features_t *classModel)
{
    accelWindow sampleWindow;
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_SIZE; ++i) {
        acquire_window(sampleWindow);
        transform(sampleWindow);
        featurize(&features, sampleWindow);

        classModel[i] = features;
    }
}
```

Training

```
void train(features_t *classModel)
{
    accelWindow sampleWindow;
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_SIZE; ++i) {
        acquire_window(sampleWindow);
        transform(sampleWindow);
        featurize(&features, sampleWindow);

        classModel[i] = features;
    }
}
```

Collect 4 x-y-z values.

```
void acquire_window(accelWindow window){
    accelReading sample;
    unsigned samplesInWindow = 0;

    while (samplesInWindow < ACCEL_WINDOW_SIZE)
    {
        accel_sample(&sample);
        window[samplesInWindow++] = sample;
    }
}
```

```
void accel_sample(accelReading* result){
    result->x = readAccelX();
    result->y = readAccelY();
    result->z = readAccelZ();
}
```

Training

```
void train(features_t *classModel)
{
    accelWindow sampleWindow;
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_SIZE; i++) {
        acquire_window(sampleWindow);
        transform(sampleWindow, &features);
        featurize(&features, &features_t);

        classModel[i] = featurize(&features, &features_t);
    }
}
```

```
#define SAMPLE_NOISE_FLOOR 10

void transform(accelWindow window)
{
    unsigned i = 0;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        accelReading *sample = &window[i];
        if (sample->x < SAMPLE_NOISE_FLOOR ||
            sample->y < SAMPLE_NOISE_FLOOR ||
            sample->z < SAMPLE_NOISE_FLOOR) {

            sample->x = (sample->x > SAMPLE_NOISE_FLOOR) ? sample->x : 0;
            sample->y = (sample->y > SAMPLE_NOISE_FLOOR) ? sample->y : 0;
            sample->z = (sample->z > SAMPLE_NOISE_FLOOR) ? sample->z : 0;
        }
    }
}
```

Filter bad values.

Extract the features: calculate mean and standard deviation of the collected data.

Training

```
void train(features_t *cl
{
    accelWindow sampleWind
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_
        acquire_window(sam
        transform(sampleWi
        featurize(&feature

    classModel[i] = fe
}
}
```

```
void featurize(features_t *features, accelWindow aWin){
    accelReading mean;
    accelReading stddev;

    mean.x = mean.y = mean.z = 0;
    stddev.x = stddev.y = stddev.z = 0;
    int i;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        mean.x += aWin[i].x; // x
        mean.y += aWin[i].y; // y
        mean.z += aWin[i].z; // z
    }

    mean.x >>= 2; mean.y >>= 2; mean.z >>= 2;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        stddev.x += aWin[i].x > mean.x ? aWin[i].x - mean.x : mean.x - aWin[i].x;
        stddev.y += aWin[i].y > mean.y ? aWin[i].y - mean.y : mean.y - aWin[i].y;
        stddev.z += aWin[i].z > mean.z ? aWin[i].z - mean.z : mean.z - aWin[i].z;
    }

    stddev.x >>= 2; stddev.y >>= 2; stddev.z >>= 2;

    unsigned meanmag = mean.x*mean.x + mean.y*mean.y + mean.z*mean.z;
    unsigned stddevmag = stddev.x*stddev.x + stddev.y*stddev.y + stddev.z*stddev.z;
    features->meanmag = sqrt16(meanmag);
    features->stddevmag = sqrt16(stddevmag);
}
```

Extract the features: calculate mean and standard deviation of the collected data.

Training

```
void train(features_t *cl
{
    accelWindow sampleWind
    features_t features;
    unsigned i;

    for (i = 0; i < MODEL_
        acquire_window(sam
        transform(sampleWi
        featurize(&feature

    classModel[i] = fe
}
}
```

```
void featurize(features_t *features, accelWindow aWin){
    accelReading mean;
    accelReading stddev;

    mean.x = mean.y = mean.z = 0;
    stddev.x = stddev.y = stddev.z = 0;
    int i;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        mean.x += aWin[i].x; // x
        mean.y += aWin[i].y; // y
        mean.z += aWin[i].z; // z
    }

    mean.x >>= 2; mean.y >>= 2; mean.z >>= 2;

    for (i = 0; i < ACCEL_WINDOW_SIZE; i++) {
        stddev.x += aWin[i].x > mean.x ? aWin[i].x - mean.x : mean.x - aWin[i].x;
        stddev.y += aWin[i].y > mean.y ? aWin[i].y - mean.y : mean.y - aWin[i].y;
        stddev.z += aWin[i].z > mean.z ? aWin[i].z - mean.z : mean.z - aWin[i].z;
    }

    stddev.x >>= 2; stddev.y >>= 2; stddev.z >>= 2;

    unsigned meanmag = mean.x*mean.x + mean.y*mean.y + mean.z*mean.z;
    unsigned stddevmag = stddev.x*stddev.x + stddev.y*stddev.y + stddev.z*stddev.z;
    features->meanmag = sqrt16(meanmag);
    features->stddevmag = sqrt16(stddevmag);
}
```

$$\text{mean} = \frac{1}{N} \sum_i x$$
$$\text{stddev} = \frac{\sum_i |x - \mu|}{N}$$

Main Program

- Recognize!

```
int activity_main()
{
    ...
    model_t model;

    while (1)
    {
        switch (mode) {
            case MODE_TRAIN_STATIONARY:
                train(model.stationary);
                break;
            case MODE_TRAIN_MOVING:
                train(model.moving);
                break;
            case MODE_RECOGNIZE:
                recognize(&model);
                break;
            default:
                break;
        }
    }

    return 0;
}
```

Recognize

```
#define SAMPLES_TO_COLLECT 128

void recognize(model_t *model)
{
    stats_t stats;
    accelWindow sampleWindow;
    features_t features;
    class_t class;
    unsigned i;

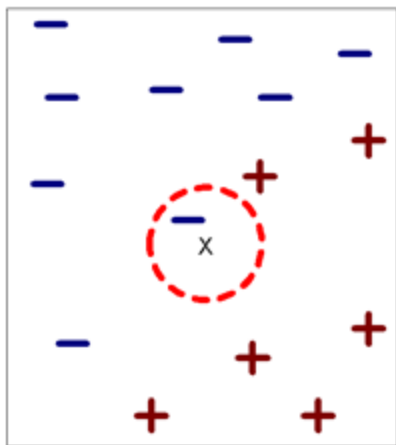
    stats.totalCount = 0;
    stats.stationaryCount = 0;
    stats.movingCount = 0;

    for (i = 0; i < SAMPLES_TO_COLLECT; ++i) {
        acquire_window(sampleWindow);
        transform(sampleWindow);
        featurize(&features, sampleWindow);
        class = classify(&features, model);
    }
}
```

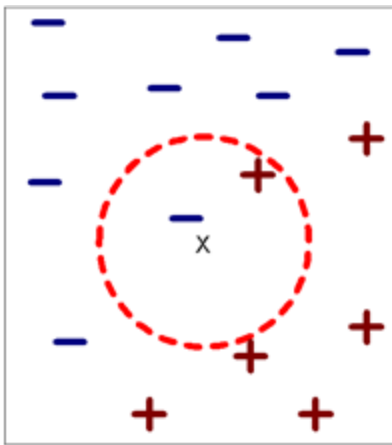
The same steps as in the training phase.

K-Nearest Neighbor Classification

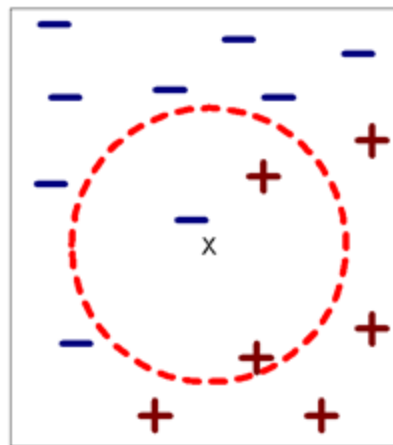
- K-nearest neighbors of a record x are data points that have the k smallest distance to x .



(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor

Recognize

```
#define SAMPLES_TO_COLLECT 128

void recognize(model_t *model)
{
    stats_t stats;
    accelWindow sampleWindow;
    features_t features;
    class_t class;
    unsigned i;

    stats.totalCount = 0;
    stats.stationaryCount = 0;
    stats.movingCount = 0;

    for (i = 0; i < SAMPLES_TO_COLLECT; ++i) {
        acquire_window(sampleWindow, model->sampleWindow);
        transform(sampleWindow, &features);
        featurize(&features, &stats);
        class = classify(&features, &stats, model);
    }
}
```

```
class_t classify(features_t *features, model_t *model){
    int move_less_error = 0; int stat_less_error = 0;
    features_t *model_features; int i;

    for (i = 0; i < MODEL_SIZE; ++i) {
        model_features = &model->stationary[i];

        long int stat_mean_err = (model_features->meanmag > features->meanmag)
            ? (model_features->meanmag - features->meanmag)
            : (features->meanmag - model_features->meanmag);

        long int stat_sd_err = (model_features->stddevmag > features->stddevmag)
            ? (model_features->stddevmag - features->stddevmag)
            : (features->stddevmag - model_features->stddevmag);

        model_features = &model->moving[i];

        long int move_mean_err = (model_features->meanmag > features->meanmag)
            ? (model_features->meanmag - features->meanmag)
            : (features->meanmag - model_features->meanmag);

        long int move_sd_err = (model_features->stddevmag > features->stddevmag)
            ? (model_features->stddevmag - features->stddevmag)
            : (features->stddevmag - model_features->stddevmag);

        if (move_mean_err < stat_mean_err) move_less_error++; else stat_less_error++;
        if (move_sd_err < stat_sd_err) move_less_error++; else stat_less_error++;
    }

    class_t class = move_less_error > stat_less_error ? CLASS_MOVING : CLASS_STATIONARY;
    return class;
}
```

Summary

- Observe using a real accelerometer and see how it runs!

